Android
Unidad 3. Listas desplegables

PROGRAMACIÓN MULTIMEDIA Y DE DISPOSITIVOS MÓVILES

2º DAM

Índice

1. Listas desplegables
2. Grids
3. Temas

1. Listas desplegables

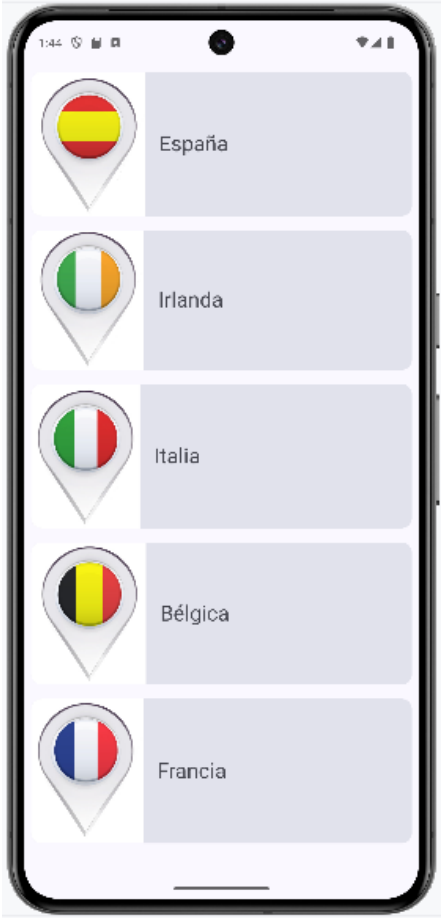
Para crear una lista desplegable (diferida) en *Jetpack Compose* utilizaremos el elemento de composición *LazyColumn* o *LazyRow*, dependiendo en la orientación en la que se dispongan sus elementos.

La diferencia con *Column* o *Row* es que éstos son estáticos, es decir contienen una cantidad fija de elementos que se cargan todos a la vez y nos podemos mover entre ellos usando las funciones de *scroll*.

LazyColum y *LazyRow* nos permite añadir elementos dinámicamente, por lo que no es perfecto para listas largas. Además nos permite desplazarnos por la lista sin añadir código extra.

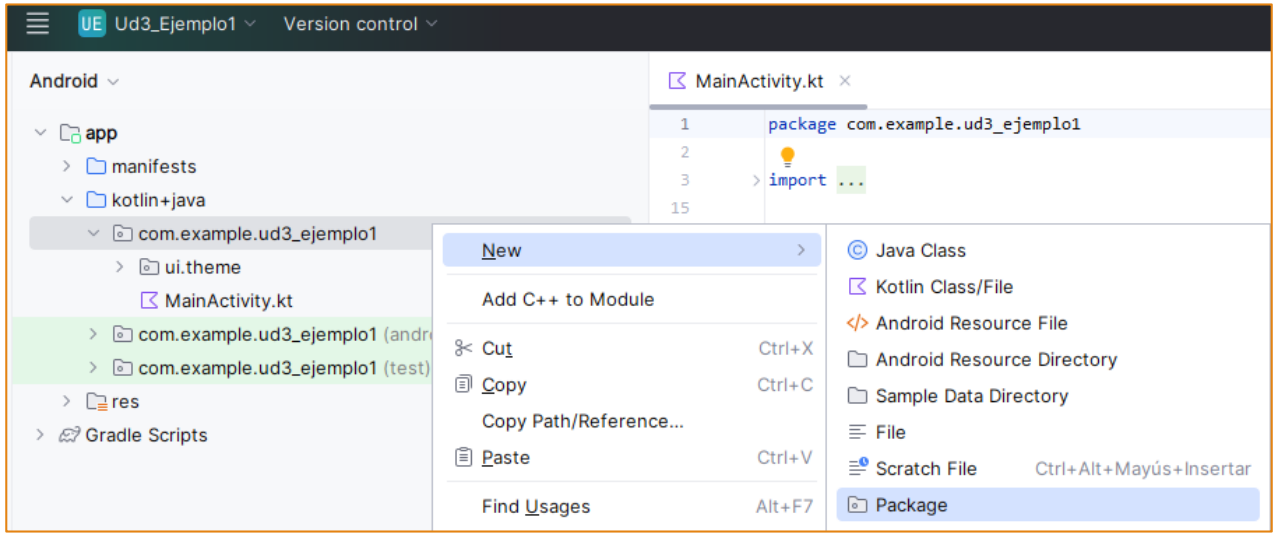
Para añadir elementos a la lista utilizaremos la función *item* para añadir un solo elemento o *ítems* para añadir más de uno.

1. Listas desplegables: Ejemplo



1. Listas desplegables: Ejemplo

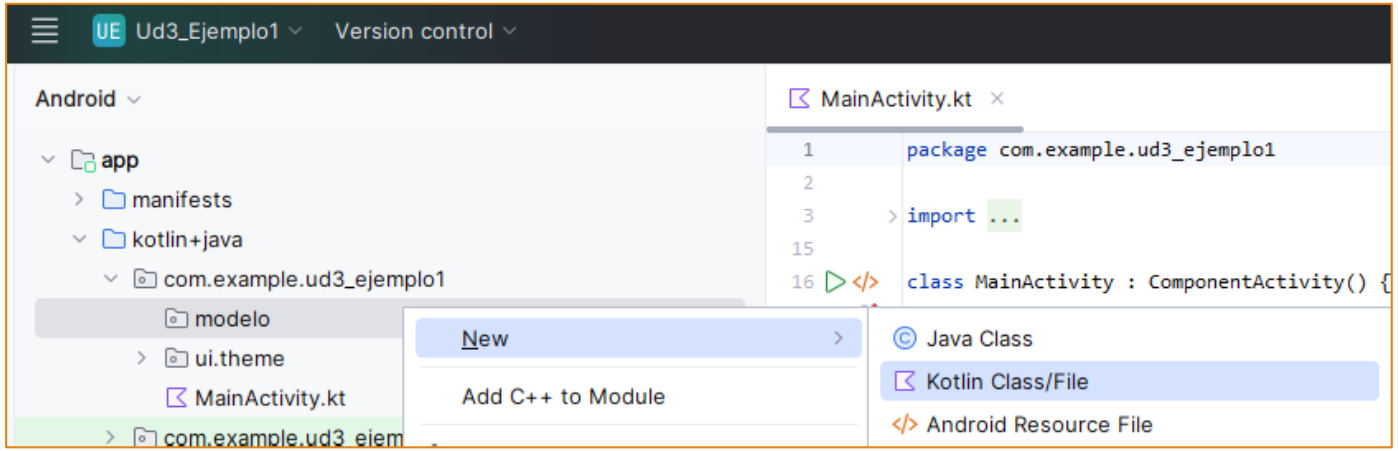
Creamos un ***paquete*** para guardar el ***modelo*** de datos.

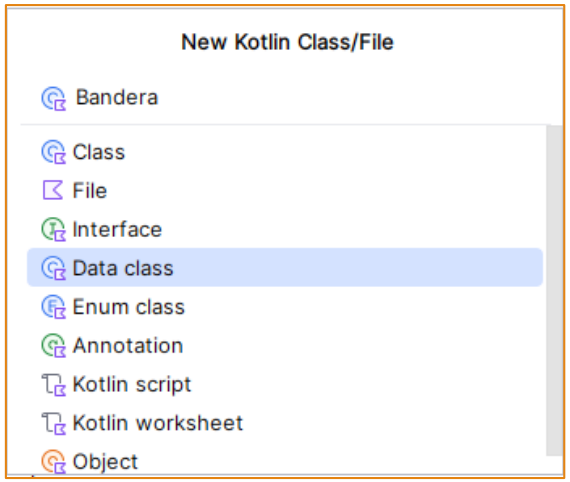




1. Listas desplegables: Ejemplo

Dentro del paquete *modelo* creamos una clase **data** llamada ***Bandera***.





1. Listas desplegables: Ejemplo

Dentro del paquete *modelo* creamos una clase **data** llamada ***Bandera***.

Bandera.kt

```kotlin
package com.example.ud3_ejemplo1.modelo

import ...

data class Bandera(
    // Necesitamos dos ids, uno para el texto de la bandera y otro para la imagen
    @StringRes val stringResourceId: Int,
    @DrawableRes val imagenResourceId: Int
)
```

@StringRes y *@DrawableRes* son anotaciones para indicar que son *ids* de recursos.

1. Listas desplegables: Ejemplo

Creamos otro paquete llamado ***datos*** y creamos una clase llamada ***Datos*** con una función que cargue los datos de las banderas.



Datos.kt

```kotlin
package com.example.ud3_ejemplo1.datos

import ...

class Datos {
    fun cargarBanderas(): List<Bandera> = listOf(
        Bandera(R.string.bandera1, R.drawable.bandera1),
        Bandera(R.string.bandera2, R.drawable.bandera2),
        Bandera(R.string.bandera3, R.drawable.bandera3),
        Bandera(R.string.bandera4, R.drawable.bandera4),
        Bandera(R.string.bandera5, R.drawable.bandera5)
    )
}
```

1. Listas desplegables: Ejemplo

Implementamos una función de composición a la que le pasamos una lista con los datos cargados y la dibujará en pantalla.

```
33  class MainActivity : ComponentActivity() {  Your Name
34      override fun onCreate(savedInstanceState: Bundle?) {  Your Name
35          super.onCreate(savedInstanceState)
36          enableEdgeToEdge()
37          setContent {
38              Ud3_Ejemplo1Theme {
39                  Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
40                      Ud3Ejemplo1App(
41                          modifier = Modifier.padding(innerPadding)
42                      )
43                  }
44              }
45          }
46      }
47  }
48
49  @Composable  Your Name
50  fun Ud3Ejemplo1App(modifier: Modifier = Modifier) {
51      ListaBanderas(
52          lista = Datos().cargarBanderas(),
53          modifier = modifier
54      )
55  }
```

1. Listas desplegables: Ejemplo

En la función *ListaBanderas* usaremos el elemento *LazyColumn* y con la función *items* recorreremos la lista creada y por cada elemento llamaremos a la función *TarjetaBandera* para que dibuje la tarjeta correspondiente con cada bandera.

```
57      @Composable  Your Name
58      fun ListaBanderas(lista: List<Bandera>, modifier: Modifier = Modifier) {
59          LazyColumn(modifier = modifier) {
60              items(lista) { bandera ->
61                  TarjetaBandera(
62                      bandera = bandera,
63                      modifier = Modifier.padding(8.dp)
64                  )
65              }
66          }
67      }
```

```
70      @Composable  Your Name *
71      fun TarjetaBandera(bandera: Bandera, modifier: Modifier = Modifier) {
72          Card(modifier = modifier) {
73              Row(modifier = Modifier
74                  .fillMaxWidth(),
75                  verticalAlignment = Alignment.CenterVertically
76              ) {
77                  Image(
78                      painter = painterResource(bandera.imagenResourceId),
79                      contentDescription = stringResource(bandera.stringResourceId),
80                  )
81                  Text(
82                      text = stringResource(bandera.stringResourceId),
83                      modifier = Modifier.padding(16.dp),
84                      style = MaterialTheme.typography.headlineSmall
85                  )
86              }
87          }
88      }
```

2. *Grids*

Compose también nos permite mostrar los elementos en una cuadrícula (*grid*) utlizatando los elementos de composición *LazyVerticalGrid* o *LazyHorizontalGrid* en función de si queremos recorrerla vertical u horizontalemente.

Los argumentos *columns* del elemento *LazyVerticalGrid* y *rows* de *LazyHorizontalGrid* nos permiten controlar cómo crear las celdas de las columnas y filas respectivamente.

Por ejemplo:

- Si sabemos el número exacto de filas o columnas del *grid* usaremos la función ***GridCells.Fixed*** a la que le pasaremos el número de filas o columnas.
- Si no lo sabemos usaremos la función ***GridCells.Adaptive*** a la que le pasaremos el tamaño mínimo de las filas o columnas del *grid*.

2. *Grids*

```
73      @Composable
74      fun GridBanderas(lista: List<Bandera>, modifier: Modifier = Modifier) {
75          LazyVerticalGrid(
76              columns = GridCells.Fixed( count: 2),
77              verticalArrangement = Arrangement.spacedBy(8.dp),
78              horizontalArrangement = Arrangement.spacedBy(8.dp),
79              modifier = modifier
80          ) {
81              items(lista) { bandera ->
82                  TarjetaBandera(
83                      bandera = bandera
84                  )
85              }
86          }
87      }
```

```
73      @Composable
74      fun GridBanderas(lista: List<Bandera>, modifier: Modifier = Modifier) {
75          LazyVerticalGrid(
76              columns = GridCells.Adaptive(minSize = 128.dp),
77              verticalArrangement = Arrangement.spacedBy(8.dp),
78              horizontalArrangement = Arrangement.spacedBy(8.dp),
79              modifier = modifier
80          ) {
81              items(lista) { bandera ->
82                  TarjetaBandera(
83                      bandera = bandera
84                  )
85              }
86          }
87      }
```

3. Temas

Material Design ofrece técnicas para trabajar con el estilo de una interfaz gráfica de tal forma que podamos modificar el tema de *Material* utilizado.

Dentro de nuestro proyecto encontraremos el tema utilizado en el fichero ***ui.theme/Theme.kt***

Los estilos más utilizados son:

- Color
- *Shapes* (formas)
- Tipografía

3. Temas: Color

Los colores utilizados en nuestro proyecto se encuentran en ***ui.theme/Color.kt***

Dentro del tema podemos encontrar los siguientes tipos de colores:

- *primary* usados para los componentes importantes de la IU como *App Bars* o botones.
- *secondary* usados para los componentes menos importantes de la IU como los *FABs*.
- tertiary usados para equilibrar los colores primarios y secundarios, o dirigir la atención hacia un elemento, como un campo de entrada.

Podemos modificar nuestros colores basándonos en la aplicación *Material Theme Builder* que nos permite exportar el tema seleccionado, pulsando en el icono  y eligiendo la opción de exportación *Jetpack Compose (Theme.kt)*. La exportación descargará un comprimido con una carpeta y dos ficheros *Theme.kt* y *Color.kt* con los que tendremos que actualizar los mismos ficheros de nuestro proyecto.

3. Temas: *Shapes*

La mayoría de las *Shapes* (formas) se basan en redondear las esquinas de los elementos utilizando la clase *RoundedCornerShape* a la que a su constructor se le pasa la cantidad en *dp* que se quiere redondear.

Podemos redondear cada esquinas de forma independiente utilizando los siguientes parámetros:

- *topStart*
- *topEnd*
- *bottomEnd*
- *bottomStart*

3. Temas: *Shapes*

La mayoría de las *Shapes* (formas) se basan en redondear las esquinas de los elementos utilizando la clase *RoundedCornerShape* a la que a su constructor se le pasa la cantidad en *dp* que se quiere redondear.

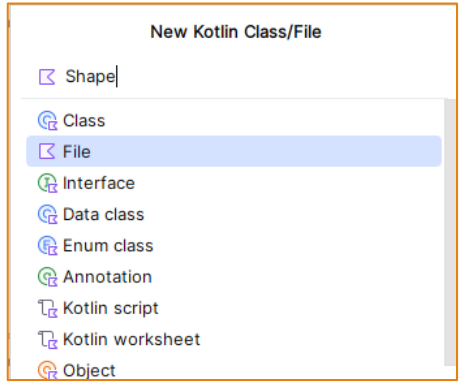
Podemos redondear cada esquinas de forma independiente utilizando los siguientes parámetros:

- *topStart*
- *topEnd*
- *bottomEnd*
- *bottomStart*

Además, en Compose existen dos formas ya definidas *RectangleShape* y *CircleShape*

3. Temas: *Shapes*

Si queremos crear nuestras propias *shapes* crearemos un fichero *Shapes.kt* dentro del paquete *ui.theme* de nuestro proyecto con una sola variable en la que podremos definir los diferentes estilos de *shapes* que existen.



Shape.kt

```kotlin
package com.example.ud3_ejemplo1.ui.theme

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Shapes
import androidx.compose.ui.unit.dp

val Shapes = Shapes(
    small = RoundedCornerShape(50.dp),
    medium = RoundedCornerShape(bottomStart = 50.dp, topEnd = 0.dp)
)
```

```kotlin
@Composable  Your Name *
fun TarjetaBandera(bandera: Bandera, modifier: Modifier = Modifier) {
    Card(modifier = modifier) {
        Row(modifier = Modifier
            .fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Image(
                painter = painterResource(bandera.imagenResourceId),
                contentDescription = stringResource(bandera.stringResourceId),
                modifier = Modifier.clip(MaterialTheme.shapes.small)
                )
            Text(
                text = stringResource(bandera.stringResourceId),
                modifier = Modifier.padding(16.dp),
                style = MaterialTheme.typography.headlineSmall
            )
        }
    }
}
```

3. Temas: *Shapes*

También podremos utilizar los *shapes* ya definidos por *Compose*:

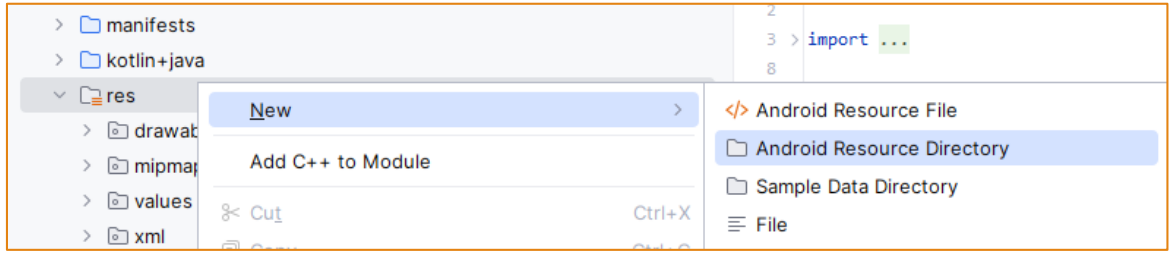
```
74      @Composable  Your Name *
75      fun TarjetaBandera(bandera: Bandera, modifier: Modifier = Modifier) {
76          Card(modifier = modifier,
77              shape = RectangleShape) {
78              Row(modifier = Modifier
79                  .fillMaxWidth(),
80                  verticalAlignment = Alignment.CenterVertically
81              ) {
82                  Image(
83                      painter = painterResource(bandera.imagenResourceId),
84                      contentDescription = stringResource(bandera.stringResourceId),
85                      modifier = Modifier.clip(CircleShape)
86                      )
87                  Text(
88                      text = stringResource(bandera.stringResourceId),
89                      modifier = Modifier.padding(16.dp),
90                      style = MaterialTheme.typography.headlineSmall
91                  )
92              }
93          }
94      }
```

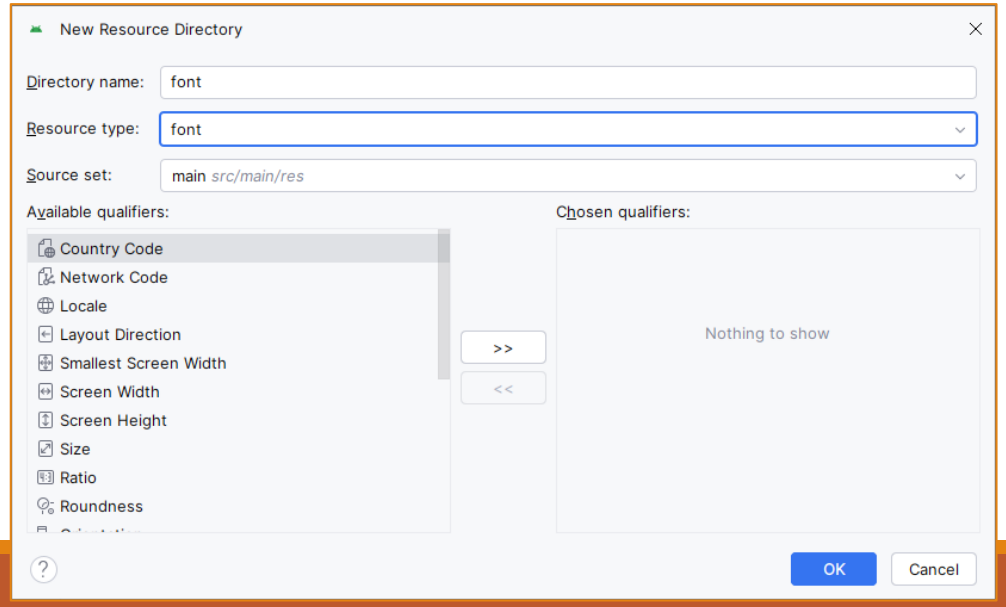
3. Temas: Tipografía

Material Desgin nos ofrece diferentes escalas de tipografía.

Para añadir una fuente nueva a nuestro proyecto tendremos que hacer click derecho sobre la carpeta ***res*** y elegir ***Android Resource Directory***. Le daremos el nombre de ***font*** y elegiremos el tipo de recurso ***font***.

Dentro de esta carpeta almacenaremos las fuentes que queramos utilizar.

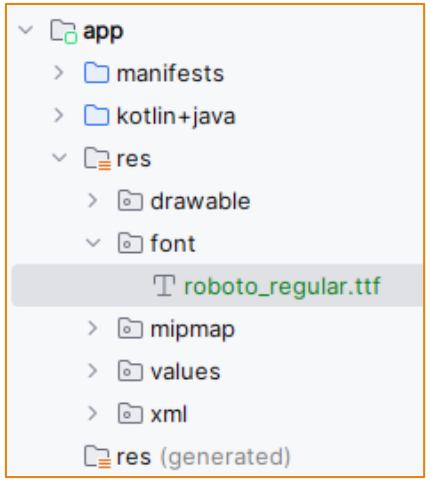




3. Temas: Tipografía

A partir de ahora podremos utilizar las fuentes de la carpeta ***font*** inicializándolas en el fichero ***ui.theme/Type.kt***

Para ello, arrastraremos la fuente a la carpeta ***font***, le cambiaremos el nombre (minúsculas y guiones bajos), creamos una nueva variable dentro del fichero ***Type.kt*** y la utilizaremos en la escala que nos interese.



Type.kt

```kotlin
package com.example.ud3_ejemplo1.ui.theme

import ...

val Roboto = FontFamily(
    Font(R.font.roboto_regular)
)

// Set of Material typography styles to start with
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    ),
    displayMedium = TextStyle(
        fontFamily = Roboto,
        fontWeight = FontWeight.Bold,
        fontSize = 20.sp
    )
```